Homework 5: Model-Based and Offline RL

CMU 10-703: Deep Reinforcement Learning (Fall 2026)

OUT: **10/26/2026**
DUE: **11/11/2026**

Instructions: START HERE

- **Collaboration policy:** You may work in groups of up to three people for this assignment. It is also OK to get clarification (but not solutions) from books or online resources after you have thought about the problems on your own. You are expected to comply with the University Policy on Academic Integrity and Plagiarism[1].
- **Late Submission Policy:** You are allowed a total of 10 grace days for your homeworks. However, no more than 2 grace days may be applied to a single assignment. Any assignment submitted after 2 days will not receive any credit. Grace days do not need to be requested or mentioned in emails; we will automatically apply them to students who submit late. We will not give any further extensions so make sure you only use them when you are absolutely sure you need them. See the Assignments and Grading Policy for more information about grace days and late submissions: https://cmudeeprl.github.io/703website_f26/
- **Submitting your work:**
  - **Gradescope:** Please write your answers and copy your plots into the provided LaTeX template (`template.tex`), and upload a PDF to the Gradescope assignment titled "Homework 5." Additionally, zip your `code/` directory into `<andrew id>.zip` and upload it to the Gradescope assignment titled "Homework 5: Code." Each team should only upload one copy of each part, and mark all teammates on **both** uploads. Regrade requests can be made within one week of the assignment being graded.
- **Setup:** Follow `code/README.md` to create a single Python environment for the whole assignment (`pip install -r requirements.txt`). All commands below are run from the `code/` directory, and every piece of code you need to write is marked with `BEGIN STUDENT SOLUTION`.

[1] https://www.cmu.edu/policies/

Introduction

This assignment has two parts. In **Part I** (Problem 1) you will implement a model-based reinforcement learning method, PETS, and then use a learned dynamics model to generate synthetic data for a model-free learner (TD3). In **Part II** (Problems 2–3) you will study the *offline* RL setting, where the agent only has access to a static dataset, and make the same TD3 implementation work there with behavior regularization. Both parts share one TD3 implementation, `common/td3.py`.

Problem	Topic	Points
1	Model-based RL with PETS	105
2	Offline RL: TD3 + behavior regularization	50
3	Conceptual questions on offline RL	50
	Total	205

Part I: Model-Based RL

Problem 1: Model-Based RL with PETS (105 pts)

In previous homeworks, you implemented REINFORCE, N-Step A2C, and DQN, which are all model-free methods. For this problem, you will implement a model-based reinforcement learning (MBRL) method called PETS, which stands for probabilistic ensemble and trajectory sampling [1]. You will find the original PETS paper to be very useful for this assignment, so we encourage you to read this paper thoroughly. An overview of MBRL with PETS is shown in Algorithm 1. The code for this problem lives in `pets/`, `run_pets.py`, and `run_synthetic.py`.

Algorithm 1 MBRL with PETS

procedure MBRL(# training iterations K_{train})
    Initialize empty data array $\mathcal{D}$ and initialize probabilistic ensemble (PE) of models.
    Sample 100 episodes from the environment using random actions and store into $\mathcal{D}$.
    Train PE for 10 steps using $\mathcal{D}$.
    repeat for K_{train} iterations:
        Sample 1 episode using MPC and latest PE and add to $\mathcal{D}$.
        Train PE for 1 step using $\mathcal{D}$.

There are 3 main components to MBRL with PETS:

1. **Probabilistic ensemble of networks:** you will be using probabilistic networks that output a distribution over the resulting state given a state and action pair.
2. **Trajectory sampling:** propagate hallucinated trajectories through time by passing hypothetical state-action pairs through different networks of the ensemble.
3. **Model predictive control:** use the trajectory sampling method along with a cost function to perform planning and select good actions.

We will go into more detail on each component below.

Part 1: Probabilistic Ensemble

You are provided starter code in `pets/model.py` that specifies the model architecture that you will be using for each member of the ensemble. In this homework an ensemble of 2 networks should be sufficient. Specifically, each network is a fully connected network with 3 hidden layers, each with 400 hidden nodes. If you have trouble running this network, a smaller network may work as well but may require additional hyperparameter tuning. The starter code also includes a method for calculating the output of each network, which will return the mean and log variance of the next state distribution conditioned on a current state and action. We recommend using a diagonal log variance matrix for models in the PE.

The loss that you should use to train each network for a minibatch of size B is the negative log likelihood of the observed next states under the predicted mean and variance from the network, conditioned on the observed current states and actions (we recommend using $B = 128$ and using Adam as the optimizer with learning rate 10^{-3}).

The training routine for the ensemble is shown in Algorithm 2. You will need to implement this in the `train_model` function in `pets/model.py`.

Algorithm 2 Training the Probabilistic Ensemble

procedure TRAIN_STEP(data $\mathcal{D}$, # networks N, minibatch size B)
  for n in $1 : N$ **do**
    Uniformly sample (with replacement) a minibatch of size B from $\mathcal{D}$.
    Take a gradient step of the loss for the sampled minibatch.

Part 2: Trajectory Sampling

For your implementation of PETS, you will use the TS1 sampling method, which is shown in Algorithm 3. Note that this algorithm only determines which networks will be used for which time steps to perform the state prediction. It is only one component of model predictive control and will be combined with the model and action optimization method in the next section.

Algorithm 3 Trajectory Sampling with TS1

procedure TS1(# networks N, # particles P, plan horizon T)
  Initialize array S of dimension $P \times T$ to store network assignments for each particle.
  for p in $1 : P$ **do**
    Randomly sample a sequence s of length T where each $s_i \in \{1, \ldots, N\}$.
    Set $S[p, :] = s$.

You can think of this trajectory sampling method as returning P particles that each represent a path of length T where at each time step, a random network in the ensemble is used to generate the next state.

Part 3: Action Selection with the Cross Entropy Method

In order to perform action selection, we need a cost function to evaluate the fitness of different state and action pairs. Defining the right cost function is often the hardest part of getting model-based reinforcement learning to work, since the action selection and resulting trajectories from MPC

depend on the cost function. Note that it is not appropriate to use the negative of the return for our problem as rewards are so sparse (we only receive rewards when the box reaches the goal). For this reason, we use surrogate costs (negative rewards) that are less sparse. For the `Pushing2D-v1` environment, you will be using the following cost function:

$$\text{cost}(\textbf{pusher}, \textbf{box}, \textbf{goal}) = d(\textbf{pusher}, \textbf{box}) + 2\, d(\textbf{box}, \textbf{goal}) + 5 \left| \frac{x_{\textbf{box}}}{y_{\textbf{box}}} - \frac{x_{\textbf{goal}}}{y_{\textbf{goal}}} \right| \tag{1}$$

where d denotes Euclidean distance and $\textbf{pusher}, \textbf{box}, \textbf{goal}$ denote the tuples of (x, y) coordinates of the pusher, box and goal at time t respectively, and x_{obj} denotes the x coordinate of an object (similar for y). Feel free to play around with other cost functions to see if you can do better.

We can now use TS1, along with the cost function, to estimate the cost of a particular action sequence $a_{1:T}$ from state s_0. Let TS be the output of TS1 for our settings of N, P, T and let $s_{(p,t)} \sim \text{model}_{TS[p,t]}.\text{predict}(s_{(p,t-1)}, a_{t-1})$; that is, the next state for a given particle is the predicted state from the model indicated by TS for the given particle and time step applied to the last state of the particle with the given action from $a_{1:T}$ (remember that we are using a probabilistic model so the output is a sample from a normal distribution). We can now calculate the cost of a state and action sequence pair as the sum of the average of the cost over all particles:

$$C(a_{1:T}, s) = \frac{1}{P} \sum_{p=1}^{P} \sum_{t=1}^{T} \text{cost}(s_{(p,t)}) \tag{2}$$

where $s_{(p,t)}$ is calculated as described above.

Finally, we can optimize the action sequence with the cross entropy method (CEM), shown in Algorithm 4. We have included an implementation of CEM in `pets/optimizers.py`, which you are free to use.

Algorithm 4 CEM

1: **procedure** CEM(population size M, # elites e, # CEM iters K_{CEM}, μ, σ)
2: **for** i in $1 : K_{\text{CEM}}$ **do**
3: Generate M action sequences according to μ and σ from a normal distribution.
4: **for** m in $1 : M$ **do**
5: Calculate the cost of $a_{m,1:T}$ according to Equation 2.
6: Update μ and σ using the top e action sequences.
7: **return:** μ

Finally: Tying It All Together

The only missing piece left to implement Algorithm 1 is line 6, i.e., how to sample an episode using MPC; this is shown in Algorithm 5. We proceed by starting with an initial μ and σ and use that as input to CEM. Then we take the updated μ from CEM and execute the action in the first time step in the environment to get a new state that we use for MPC in the next time step. We then update the μ that is used for the next timestep to be the μ from CEM for the remaining steps in the plan horizon and initialize the last time step to 0. Finally, we return all the state transitions gathered so that they can be appended to $\mathcal{D}$. Please use a planning horizon of $T = 5$.

Algorithm 5 Generating an episode using MPC

1: **procedure** MPC(env, plan horizon T)
2: transitions = []
3: s = env.reset()
4: $\mu = 0$, $\sigma = 1$
5: **while** not done **do**
6: μ = CEM(200, 20, 5, μ, σ)
7: $a = \mu[0, :]$
8: $s' = \text{env.step}(a)$
9: transitions.append(s, a, s')
10: $s = s'$
11: $\mu = \mu[1 : T]$.append(0)
12: **return:** transitions

Note that on line 11, $\mu[1 : T]$ uses list indexing following Python indexing conventions (i.e., using zero indexing and not including T). For this problem, please respond to the prompts below.

1.1 Model-Based Predictive Control (25 pts)

Before you begin to implement, we recommend going through how `run_pets.py` works in a top-down manner, to see how each component of the code works together. For all the questions in 1.1, the experiments are in `test_cem_gt_dynamics` in `run_pets.py`; run them with `python run_pets.py -question 1.1`.

1. **(10 pts)** Implement the `predict_next_state_gt` function in `pets/mpc.py`. Then test CEM (without MPC) with the ground-truth dynamics on the `Pushing2D-v1` environment. The CEM policy will also be used later for planning over a learned dynamics model. All the hyperparameters are provided in the code. Report the percentage of successes over 50 episodes.
2. **(10 pts)** Instead of CEM, plan with random action sequences where each action is generated independently from the same initial distribution as CEM, $\mathcal{N}(0, 0.5^2 I)$, where I is an identity matrix. Use the same number of trajectories for planning as CEM, i.e., for each state, sample $M \cdot K_{\text{CEM}}$ trajectories of length T and pick the best one. Report the percentage of successes over 50 episodes, both without and with MPC. How does its performance compare to CEM?
3. **(5 pts)** Which algorithm (MPC vs. open-loop control) would perform better on what environments? Why? Discuss the pros and cons of MPC. The script also compares CEM with and without MPC on `Pushing2D-v1` and on `Pushing2DNoisyControl-v1` (which adds noise to the actions), which you may use to support your answer.

1.2 Single Probabilistic Network (30 pts)

Train a single probabilistic network on transitions from 1000 randomly sampled episodes (`python run_pets.py -question 1.2`). The loss to train each network is the negative log likelihood of the actual resulting state under the predicted distribution of the network. Specifically, given state transition pairs (s_t, a_t, s_{t+1}), we want to minimize the negative log probability of s_{t+1} under the Gaussian distribution $\mathcal{N}(\mu_\theta(s_t, a_t), \Sigma_\theta(s_t, a_t))$, where μ_θ and Σ_θ are outputs of the network. Implement `get_loss` and `train_model` in `pets/model.py`.

1. **(4 pts)** Plot the loss vs. number of iterations trained for the single network.
2. **(10 pts)** Combine your model with planning using randomly sampled actions. In `pets/mpc.py`, implement `predict_next_state_model` (vectorizing this will make a big difference). Evaluate the performance of your model when planning using a time horizon of 5 and 1000 possible action sequences. Report the percentage of successes on 50 episodes.
3. **(10 pts)** Combine your model with planning using CEM. Evaluate the performance of your model when planning using a time horizon of 5, a population of 200, 20 elites, and 5 CEM iterations. Report the percentage of successes on 50 episodes.
4. **(6 pts)** Which planning method performs better, random or CEM? How did MPC using this model perform in general? When is the derived policy able to succeed and when does it fail?

1.3 MBRL with PETS (15 pts)

(This section takes about 30 minutes to generate the required results: `python run_pets.py -question 1.3`.)

1. **(10 pts)** Run Algorithm 1 for 500 iterations (i.e., collect 100 initial episodes and then 500 episodes using MPC). Plot the loss vs. the number of iterations trained.
2. **(5 pts)** Every 50 iterations, test your model with MPC on 20 episodes and report the percentage of successes. Plot this as a function of the number of iterations of PETS.

1.4 Training with Synthetic Data (35 pts)

Now, instead of using the model for planning, we will use it to generate synthetic data for training model-free RL algorithms. Download the pretrained ensemble `model.pt` (similar to the one trained in 1.3.1) from here into `code/`. Implement `get_synthetic_transition` in `run_synthetic.py`, which uses the model to predict a new state given the current state and action, and then run `python run_synthetic.py -question 1.4.1` (and `1.4.2`, `1.4.3`). Note that this takes a while (around 1 hour in total for the reference solution) because the results are quite noisy, so we need to run multiple seeds.

1. **(10 pts)** Train a model-free RL algorithm, TD3 (already implemented in `common/td3.py`), using a mix of synthetic and real environment data. Ablate over the proportion of synthetic vs. real data, and plot the success rate of the trained policies as a function of the data ratio (Model : Real Environment) $\in$ {0:1, 0.2:0.8, 0.5:0.5, 0.8:0.2, 1:0}. Discuss the impact of increasing model-generated data on policy performance and give an explanation.
2. **(10 pts)** Instead of generating rollouts from the initial state, use an intermediate state of a real environment trajectory as the starting point. From the same starting point, ablate over different model rollout lengths: 2, 4, 8 steps. Plot the success rate vs. the different model rollout lengths. Discuss how the model rollout length influences the success rate and give an explanation.
3. **(10 pts)** Compute the model error (difference between predicted and actual states) and plot it against the model rollout length. Does increasing the rollout length amplify model error?
4. **(5 pts)** Under which scenarios would you prefer MBRL to policy gradient methods?

Part II: Offline RL

In this part we will explore how we can make value-based RL algorithms (in particular TD3, the same implementation used in Problem 1.4) work in the *offline* RL setting: when we have access to a static dataset of environment interactions collected with some behavior policy, and no online access to the environment for learning. In particular, we will implement behavior regularization. All experiments in this part are run with `run_offline.py`.

Problem 2: TD3 + Behavior Regularization (50 pts)

In the offline RL setting we want to learn a policy given a static dataset of previous interactions with the environment $\mathcal{D} = \{(s, a, r, s')\}$ instead of learning by directly interacting with the environment. Our first step will be understanding what goes wrong if we simply keep using the same algorithms as before.

2.1 Loading the Offline Dataset into the Replay Buffer

We would like to run the critic and policy updates from TD3 using data uniformly sampled from the offline dataset (as opposed to from the online replay buffer normally used for TD3).

The offline dataset we are going to use comes from Minari, an offline RL tasks library. In particular, we are going to use `mujoco/hopper/medium-v0`, a classic offline RL task. In `offline/data.py`, implement `get_buffer_and_environments_for_task`, which loads data from the offline dataset into the replay buffer, and creates the environment in which we will evaluate the policies we train.

Note: for this assignment, we will not collect any online data into the replay buffer (i.e., the only data in the replay buffer will come from the static offline dataset). Therefore, the size of the replay buffer can simply be the number of timesteps in the offline dataset.

2.2 Running TD3 without Any Offline Learning Modifications

Run vanilla TD3 on the `hopper-medium-v0` task with the default hyperparameters, and plot the resulting learning curves, including evaluation returns and predicted Q-values for dataset state-action pairs.

In our reference implementation, we get evaluation returns of around 10 and Q-values of more than 3000 by step 50k. These trends of returns and Q-values give us a clue about what is going wrong here. **Question:** what insights can we draw from evaluation returns being low but predicted Q-values being high?

Command (about 30 minutes):

```
python run_offline.py --total_steps 100000
```

2.3 Behavior Regularized TD3

Next, add a behavior-cloning loss to the TD3 policy objective. The policy objective from vanilla TD3 (which comes from DDPG) is given by:

$$\mathcal{L}_\pi^{(\text{DDPG})} = -\mathbb{E}_{s\sim\mathcal{D}}\left[Q_1(s, \mu_\theta(s))\right]. \tag{3}$$

We are going to modify it by adding a behavior-cloning term to encourage the policy to stay close to the actions seen in the offline dataset:

$$\mathcal{L}_\pi^{(\text{DDPG + BC-Reg})} = -\mathbb{E}_{s,a\sim\mathcal{D}}\left[Q_1(s,\mu_\theta(s))\right] + \alpha_{\text{reg}}\,\mathbb{E}_{s,a\sim\mathcal{D}}\left[\|a-\mu_\theta(s)\|_2^2\right]. \tag{4}$$

Implement BC regularization in `TD3.update` in `common/td3.py`, and train an agent with $\alpha_{\text{reg}} = 10$:

```
python run_offline.py --bc_regularization_weight 10
```

Plot the same learning curves as in 2.2. Our reference implementation gets evaluation returns of around 500 at 50k steps.

2.4 Conceptual Questions about Picking α_{reg}

A key choice when implementing DDPG + behavior regularization is the choice of the BC-regularization coefficient α_{reg}. Answer the following questions:

1. The effect of the value of α_{reg} is environment and dataset dependent. For which kinds of **datasets** will we expect **larger** values of α_{reg} to work **better**? And conversely, for which datasets will we expect **lower** values of α_{reg} to work better?
2. We would like a systematic way of choosing α_{reg}. Let's say that you first complete a training run with $\alpha_{\text{reg}} = 1.0$, and you have access to any metric you require from this training run; how would you choose a reasonable value of α_{reg} if you could only run a single more run?

Problem 3: Conceptual Questions on Offline RL (50 pts)

(i) **Select all that apply and justify** regarding the offline RL methods that apply regularization on the value function for training. For this question, consider three algorithms:

- Conservative Q-Learning: as we discussed in class, this algorithm applies a regularizer to minimize large Q-values during training.
- Reward penalty: similar to offline model-based RL methods, we penalize the reward value at *each* state-action pair by subtracting an uncertainty estimate from it.
- Policy constraint penalty: this approach subtracts $D_{\text{KL}}(\pi_\theta, \pi_\beta)$ from the policy update and the value function update.

Select all that apply from below:

1. Conservative Q-learning is likely to perform better than policy constraints on tasks with offline datasets that exhibit higher coverage.
2. Reward penalty should perform better than conservative Q-learning methods because penalizing the Q-function at each unseen action will lead to more conservatism than using uncertainty estimates to penalize the reward function.
3. Conservative Q-learning is addressing problems of over-conservative behavior with policy constraint penalty methods.

(ii) In class, we studied several methods for offline RL based on policy constraints. Recall that there were two types of methods: one based on distributional constraints (e.g., optimize the Q-function subject to the KL-divergence between the learned policy and the behavior policy)

and one based on support constraints (e.g., optimize the Q-function subject to a support constraint between the learned policy and the behavior policy). Describe a scenario where distribution constraint methods would be preferable to support constraints. *Hint:* in class, we discussed when support constraint methods will be better. Think about when that is not the case and answer this question.

(iii) **Select all that apply and justify** regarding the performance of the model-based offline RL methods that we studied in class.

1. Model-based offline RL methods are generally better than model-free offline RL methods (that do not use a dynamics model), given any offline dataset.
2. Using distributional policy constraints within model-based offline RL methods can be helpful when the distribution of the offline dataset is **quite narrow**.
3. Using distributional policy constraints within model-based offline RL methods can be helpful when the distribution of the offline dataset is **quite broad and exhibits high coverage**.

(iv) Given a fixed static offline dataset of state-action-next state-reward transitions, why can offline RL (not filtered imitation learning) perform better than imitation learning? Give a qualitative example of a dataset where offline RL is expected to outperform imitation and one where imitation is expected to outperform offline RL.

Feedback

Feedback: You can help the course staff improve the course by providing feedback. What was the most confusing part of this homework, and what would have made it less confusing?

Time Spent: How many hours did you spend working on this assignment? Your answer will not affect your grade.

References

[1] Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. *arXiv preprint arXiv:1805.12114*, 2018.

[2] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-learning for offline reinforcement learning. *arXiv preprint arXiv:2006.04779*, 2020.